



[Productive](#) is an end-to-end professional services automation platform.

For example, a typical digital agency charges its clients for the time employees spend working on client projects. This is precisely why the employees of such an agency must record exactly how long, and on which day, they worked on which project. This process is called "time tracking" and is a basic part of Productive.

Task

Create a client-side web application called **Productive Time Tracker**. Through the application, the user should be able to manage time entries for the given day. Each time entry contains the following data: who worked, the duration, on which day, and a multiline description of the work done. New time entries can be created, existing ones should be listed, and they can be edited or deleted.

Include a technical specification as a Markdown document in the repository. The specification should describe the chosen frontend architecture, the main UI components, how the application communicates with the Productive API, and any important implementation decisions or trade-offs. If you encounter ambiguity in the assignment, make a reasonable assumption and document it in the specification.

The expected time to research, form a specification, and implement a solution is **10 hours**.

Implementation requirements

This web (JS+HTML+CSS) application must be connected to the existing Productive API for retrieving and managing organization and time entry data. The choice of technology is completely optional and it's allowed to use any JS frameworks such as React, Angular, Vue, Ember or not to use any JS framework at all. No server-side technology may be used. Include a README with setup and run instructions.

You're **encouraged** to follow your usual workflow and use a **coding agent** to implement the solution.

The visual style of the application is up to your own taste, but it should look like something you would ship to a client. It must be usable on a lower resolution (mobile) device.

To complete the task, you need to do the following:

1. Register a test [Productive account](#)
2. In the application settings, on the "API integrations" screen, find the necessary data to connect to the Productive API.

You may freely use this test account to inspect and reverse engineer the existing time tracking feature, including its UI and API communication.

Features that must be implemented are:

1. A login screen
2. A screen listing time entries for a selected date
3. Adding a new time entry for a chosen date
4. Editing an existing time entry
5. Deleting an existing time entry

Regarding the login screen, the user can enter an API token and organization ID. Credentials must be persisted so a page refresh keeps the user logged in, and a logout action must clear them. For the sake of simplicity, the form for a new time entry can only have the fields for duration, date, and work description. The person relationship is not entered by the user, but should be dynamically set to the current person.

The result of the task should be a link to the public git repository.

Evaluation

We will evaluate the frontend architecture and code structure, API integration and efficiency, the correctness and completeness of the listed requirements, the technical specification and the UI/UX quality. A submission that just implements the CRUD operations without paying attention to the other important factors will be viewed as incomplete.

Model architecture on the Productive API

The documentation for the Productive API can be found [here](#). To complete this task, you need to understand some concepts about the Productive models on the API. Each model has its own API endpoint with support for CRUD operations.

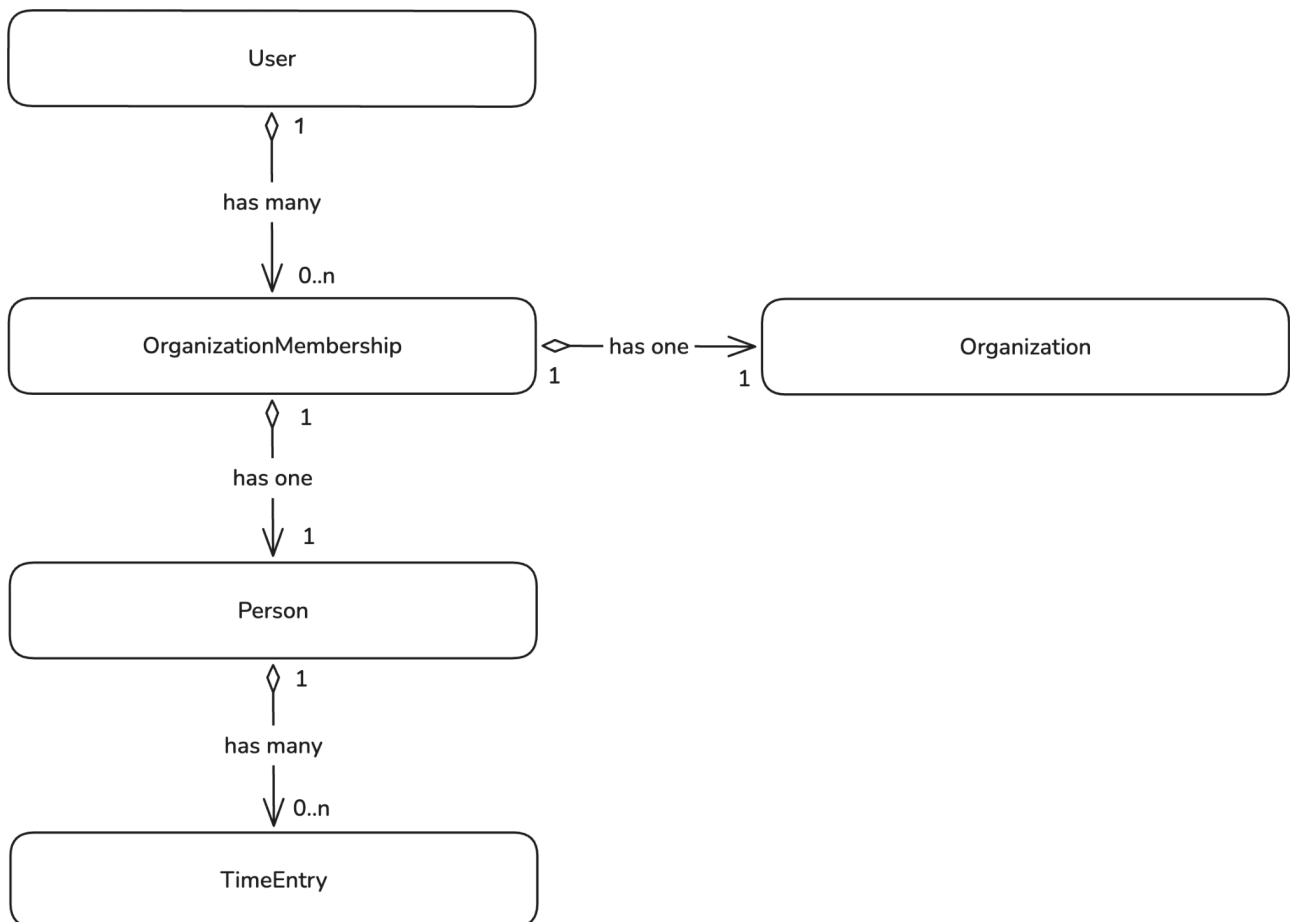
Concepts you need to understand in the context of this task are (the models are **bolded**):

- **OrganizationMembership** links a **User** to an **Organization**
- **OrganizationMembership** has a one-to-one connection to **Person**, which then connects to other models
- Each **TimeEntry** belongs to a **Person**
- Other relations on **TimeEntry** are irrelevant

The flow of a basic time tracking feature interaction is listed below:

- With the token, all **OrganizationMembership** records belonging to that user can be retrieved (GET /organization_memberships)
- **OrganizationMembership** belonging to the **Organization** with the entered ID is found among the results. From that **OrganizationMembership**, the **Person** record needed to manage **TimeEntry** records is saved
- All **TimeEntry** records for a specific date that belong to that person are listed (GET /time_entries with filters person_id, after and before)
- The amount of time worked (in minutes), a multiline description of work done, and the date are written on the form. The form is then submitted and a new **TimeEntry** record is created (POST /time_entries)

- **TimeEntry** record is edited (PATCH /time_entries)
- **TimeEntry** record is deleted (DELETE /time_entries)



User Story 1 – View time entries for a day

As a user

I want to review the time entries I logged on a particular day.

So that

I can check what I recorded on a selected date.

Acceptance criteria

- The application retrieves entries for the selected date (defaulting to today) from the Productive API when opened.
- Only entries belonging to the current user are shown.
- The user can change the selected date and the list updated accordingly.
- Entries show:
 - duration
 - description (multiline)
 - date
- If there are no entries, an empty state is displayed.
- If loading time entries fails, an appropriate error is displayed.

User Story 2 – Add a new time entry

As a user

I want to record work I have completed.

So that

My work is tracked.

Acceptance criteria

- The user can enter the duration, description and date.
- Submitting creates a new time entry.
- The list updates after successful creation.
- Validation and API errors are shown if creation fails.

User Story 3 – Edit a time entry

As a user

I want to correct an existing time entry.

So that

My recorded work stays accurate.

Acceptance criteria

- Any listed entry can be edited.
- Should exist in its own route.
- Editing can change duration, description, and date.
- Saving updates the entry via the API.
- The list reflects the changes after a successful update.
- Errors are shown if the update fails.

User Story 4 – Delete a time entry

As a user

I want to remove an incorrect time entry.

So that

My recorded work remains accurate.

Acceptance criteria

- Every entry can be deleted.
- User has to confirm the action.
- Successful deletion removes it from the list.
- Failed deletion displays an error.

For additional questions about the task, contact us by email.